

Audit complet du fonctionnement de l'application

Architecture, parcours, modules, packs, secteurs, permissions, données, sécurité, paiements, déploiement et points de vigilance.

Date de l'audit : 22 septembre 2026

Périmètre : dépôt MAXIMUS, frontend React/Vite, API Laravel/PostgreSQL, artefacts associés, documentation et tests disponibles.

Nature : audit fonctionnel et technique fondé sur le code et la documentation. Ce document n'est pas un test d'intrusion ni une certification de conformité.

Sommaire

1. Résumé exécutif
2. Périmètre, méthode et limites
3. Architecture générale et runtime actif
4. Modèle métier : catalogue, modules, packs et secteurs
5. Règles d'accès : entreprise, unité, rôle, fonctionnalité, action
6. Parcours complet d'une entreprise
7. Fonctionnement des espaces et de la navigation
8. Modules métier détaillés
9. E-commerce, boutique publique et PWA
10. Transport public / Taxi
11. Paie, paiements, portefeuille et commissions
12. Données, API et persistance
13. Sécurité, isolation et installations dédiées
14. Déploiement et exploitation
15. Tests, état de confiance et limites observées
16. Constats priorités et plan recommandé
17. Référentiel des fichiers et routes

1. Résumé exécutif

MAXIMUS est un ERP SaaS multi-entreprises en français. Son modèle central est : un catalogue publié propose des modules et des packs ; une entreprise sélectionne son périmètre ; ses unités composent ce périmètre ; les rôles distribuent des permissions ; les employés voient ensuite une navigation et des actions bornées par ces niveaux.

Conclusion générale : le socle Laravel/PostgreSQL contient les protections importantes de périmètre, de session, de module et d'action. Le frontend React/Vite expose une couverture fonctionnelle large et les tests de permissions sont nombreux. L'application est exploitable comme produit intégré, mais plusieurs domaines doivent encore être considérés comme en consolidation : abonnement réel, couverture E2E des parcours publics et financiers, coexistence Express/Drizzle-Laravel, stockage de médias en base64, absence visible de MFA/réinitialisation de mot de passe et absence de planification de maintenance.

Ce qui est fort

- Isolation tenant côté serveur : le `companyId` du navigateur ne peut pas élargir le périmètre d'un acteur non administrateur.
- Chaîne de permissions cohérente : entreprise → unité → rôle → fonctionnalité → action.
- Contrôles de paiement, QR Présences, sessions et accès aux modules protégés côté Laravel.
- Catalogue publié séparé du brouillon d'administration.
- Déploiement Render avec PostgreSQL, migrations et stockage persistant des fichiers numériques.

Ce qui doit rester sous contrôle

- Express/Drizzle et Laravel/PostgreSQL coexistent : Laravel est actif, Express reste une couche de comparaison et de migration.
- Les abonnements sont surtout modélisés côté frontend/app-state et ne pilotent pas encore l'accès serveur.
- Commerce interne conserve un repli local/prototype.
- Les images sont parfois persistées en base64, avec un risque de volumétrie et de réponse lourde.
- La couverture navigateur complète des paiements, PWA, GPS, uploads et isolation des domaines n'est pas démontrée.

Lecture rapide du produit

Catalogue publié → demande d'entreprise → validation MAXIMUS → modules actifs → unités → rôles
→ employés → navigation et actions autorisées

Sources principales : README.md:1-50, 319-370, docs/technical.md:1-18, 158-190, artifacts/api-server/laravel/app/Support/ModuleCatalog.php, middlewares Laravel et tests Feature.

2. Périmètre, méthode et limites

2.1 Éléments examinés

- Frontend artifacts/maximus : routes, pages, composants, adaptateurs API, catalogue, permissions, tests et PWA.
- Backend artifacts/api-server/laravel : routes, middlewares, contrôleurs, services, migrations et tests Feature.
- Couche Express/Drizzle existante dans artifacts/api-server/src et lib/db, pour identifier la frontière historique.
- Documentation README.md, docs/technical.md, guides d'installation et audits existants.
- Configuration Render, workflows Replit et tests exécutables disponibles.

2.2 Méthode

1. Cartographie des artefacts, routes, migrations, tests et commandes.
2. Lecture des règles de catalogue et d'autorisation côté serveur, puis comparaison avec le routage frontend.
3. Reconstitution des parcours de bout en bout : inscription, approbation, organisation, modules, boutique, paiements, Transport, Présences et Paie.
4. Recherche des écarts de persistance, d'isolation, de sécurité et de couverture de test.
5. Vérification de cohérence par les tests et le build déjà disponibles dans l'environnement.

Limites : aucun secret n'a été lu ; aucune modification de production, aucun paiement réel, aucune suppression de donnée et aucune requête de base de production n'ont été exécutés pour cet audit. Les risques signalés sont des constats de code ou des zones non prouvées par les tests disponibles, pas des vulnérabilités exploitées.

3. Architecture générale et runtime actif

3.1 Monorepo

Élément	Responsabilité	Statut
artifacts/maximus	Application React + TypeScript + Vite : écrans MAXIMUS, entreprises, employés, boutique et Transport public.	Actif
artifacts/api-server/laravel	API Laravel, middleware, contrôleurs, services, migrations et runtime PostgreSQL.	Autorité déployée
artifacts/api-server/src + lib/db	Express 5, Drizzle et schéma partiel : couche historique conservée pendant la migration et la preuve de parité.	Legacy / comparaison
artifacts/maximus-preparation-deck	Présentation de préparation du produit.	Artefact séparé
artifacts/mockup-sandbox	Prévisualisation isolée de composants.	Outil design

3.2 Démarrage et circulation d'une requête

Navigateur → Vite / frontend → préfixe /api → Laravel → middleware session/tenant/module → contrôleur/service → PostgreSQL

- src/main.tsx monte React, installe l'ErrorBoundary et nettoie le thème de tenant avant restauration.
- src/App.tsx orchestre la session, le cache frontend, la navigation et le chargement paresseux des pages.
- src/routes/app-routes.tsx sépare le routeur MAXIMUS du routeur entreprise.
- vite.config.ts exige PORT et BASE_PATH, et configure le proxy API.
- Le runtime Render cible Apache/PHP/Laravel, applique les migrations puis provisionne le compte MAXIMUS central ou une installation dédiée.

Règle opérationnelle : une correction de sécurité, d'isolation ou de permission doit être faite d'abord dans Laravel. Le frontend peut masquer une route, mais il ne constitue jamais la frontière d'autorisation.

4. Modèle métier : catalogue, modules, packs et secteurs

4.1 Définitions

Objet	Définition fonctionnelle	Où il vit principalement
Module	Grand domaine activable pour une entreprise : Commerce, E-commerce, Stocks, Présences, Paie ou Transport.	Catalogue Laravel, accès entreprise, navigation frontend.
Fonctionnalité	Sous-fonction d'un module : catalogue, commandes, inventaire, pointage, virements, etc.	Registre catalogue et permissions détaillées.
Pack	Composition prédéfinie d'un module et de ses fonctionnalités, éventuellement avec des permissions.	ModuleCatalog.php, sélection d'inscription et accès entreprise.
Secteur	Profil d'activité proposant une combinaison de modules et fonctionnalités adaptées.	Presets secteur dans le catalogue/app-state.
Accès entreprise	Ligne persistée qui dit si un module est ACTIF, BETA, MAINTENANCE ou INACTIF, avec son périmètre de fonctionnalités.	maximus_company_modules.
Rôle	Ensemble de permissions donné à une unité et à ses employés.	App-state organisation + compte serveur.

4.2 Loi de sélection

1. Le module doit être publié. Un module retiré, inconnu ou non publié est rejeté.
2. Un pack doit appartenir au module sélectionné et être encore publié.
3. Les fonctionnalités demandées doivent être des identités canoniques connues du module.
4. Une fonctionnalité sélectionnée par un pack doit appartenir à ce pack, sauf sélection explicite sans pack.
5. Les permissions ne peuvent porter que sur les fonctionnalités sélectionnées et utilisent les actions **voir**, **créer** et **modifier**.
6. Une sélection vide explicite reste vide : elle ne signifie pas « tout le pack ».
7. Les dépendances sont affichées et contrôlées dans le catalogue, mais la fermeture automatique des dépendances doit rester vérifiée lors de chaque évolution.

4.3 Publication et brouillon

Brouillon catalogue → validation MAXIMUS → catalogue publié → inscription et édition entreprise

Le parcours public d'inscription lit le catalogue publié, jamais un brouillon. Les overrides publiés et les modules personnalisés doivent être normalisés avant affichage. Une modification d'un catalogue déjà utilisé ne doit pas réactiver silencieusement une fonctionnalité retirée.

Références : app/Support/ModuleCatalog.php:10-327, 351-491, 557-597, 666-724, 878-920, AppStateController.php, OnboardingAnalysisService.php, docs/audits/company-catalogue-registration-audit.md.

5. Packs, secteurs et composition des accès

5.1 Packs intégrés par module

Module	Packs ou familles observés	Finalité
Gestion commerciale	Consultation commerciale, Gestion commerciale, Employé commercial, Manager commercial.	Clients, ventes, produits, fournisseurs, achats, caisse, rapports et activité.

E-commerce	Catalogue en ligne, Gestion e-commerce, Vente numérique, Ventes physiques et numériques, Location & réservation, Supervision boutique, Employé, Manager.	Catalogue public, commandes, clients, numérique, location, livraison, finances et paramètres.
Stocks	Consultation, Gestionnaire de stock, Responsable de stock, Employé, Manager.	Articles, entrées, sorties, demandes, inventaires, référentiels et réglages.
Présences	Consultation, Gestionnaire, Responsable, Employé, Manager.	Tableau de bord, pointage, absences, horaires, congés, historique et rapports.
Paie	Consultation paie, Gestionnaire de paie, Responsable paie, Employé, Manager.	Bénéficiaires, préparation, validation, virements, solde et historique.
Transport	Packs Taxi consultation, gestion, supervision, employé et manager selon le catalogue courant.	Courses, chauffeurs, véhicules, historique, suivi GPS et paramètres.

5.2 Loi des secteurs

Un secteur est un **profil de proposition**, pas une permission magique. Il peut proposer des modules et des fonctionnalités. Après sélection, l'accès effectif est matérialisé par les accès de l'entreprise, puis réduit par l'unité, le rôle et l'acteur.

- Les secteurs retirés ne doivent plus apparaître dans le catalogue d'inscription.
- Une fonctionnalité dépendante doit être accompagnée de son prérequis ou présentée comme impossible à sélectionner.
- Un secteur ne doit jamais donner à un employé un droit supérieur au plafond de l'entreprise.
- Un secteur ne remplace pas le contrôle serveur : il prépare une configuration, puis Laravel la renormalise à la création et à l'approbation.

Point d'attention : l'audit du code signale que la fermeture automatique des dépendances n'est pas entièrement portée par la normalisation serveur. Toute modification du catalogue doit donc vérifier explicitement les dépendances dans l'inscription, l'approbation, l'édition et l'autorisation runtime.

6. Règles d'accès et gouvernance

6.1 La chaîne de contrôle

Entreprise → unité/secteur → rôle → sous-fonctionnalité → action

Le périmètre le plus restrictif gagne. Un rôle ne peut pas réactiver un module retiré de l'entreprise ou de son unité. Le serveur reprend cette règle ; la visibilité frontend est une conséquence, pas la source d'autorité.

6.2 Acteurs

Acteur	Ce qu'il peut faire	Limite principale
Administrateur MAXIMUS	Gérer les entreprises, catalogue, accès modules, installations, paramètres plateforme, contrôle et opérations centrales.	Accès transversal ; chaque endpoint central doit néanmoins vérifier son intention.
Administrateur entreprise	Gérer son entreprise, ses unités, rôles, employés, modules autorisés et paramètres métier selon le catalogue.	Une seule entreprise ; ne peut pas élargir le plafond publié ou central.
Manager de secteur	Gérer les employés, rôles et actions dans son secteur et ses descendants autorisés.	Pas d'accès aux autres unités ; ne peut distribuer que des permissions qu'il possède.
Employé	Utiliser les fonctionnalités et actions attribuées à son rôle.	Ne peut pas gérer les comptes, l'organisation ou les périmètres hors de son identité.

Client boutique	Compte distinct : panier, commandes, adresses, favoris, téléchargement numérique, suivi et paiement.	Ne possède aucun accès à l'espace interne MAXIMUS.
-----------------	--	--

6.3 Isolation tenant

- `AuthenticateMaximus` valide la session, l'utilisateur actif et l'installation.
- `ResolveCompanyContext` force l'entreprise de l'acteur non central ; un `companyId` client différent est rejeté.
- `EnsureModuleEnabled` refuse un module inconnu, inactif ou en maintenance.
- `ModuleAuthorization` vérifie fonctionnalité et action.
- Les contrôleurs ajoutent les restrictions métier : secteur, employé, propriété de la ressource, état de l'opération.

Références : `app/Http/Middleware/AuthenticateMaximus.php`, `ResolveCompanyContext.php`, `EnsureModuleEnabled.php`, `Support/ModuleAuthorization.php`, `CompanyAuthorization.php`.

7. Parcours complet d'une entreprise

7.1 Inscription publique ou demande manuelle

1. Le frontend récupère le catalogue publié.
2. L'utilisateur choisit un secteur, des modules, des packs et éventuellement des fonctionnalités individuelles.
3. Le serveur normalise les identifiants, rejette les modules non publiés, les packs incohérents et les permissions hors périmètre.
4. La demande est créée avec une entreprise en attente et un mot de passe haché ; le mot de passe en clair n'est pas stocké.
5. La version intelligente peut analyser une description, mais l'IA propose seulement : elle ne crée ni n'active directement un module.

7.2 Approbation MAXIMUS

1. MAXIMUS vérifie que la demande est encore en attente.
2. La sélection est renormalisée contre le catalogue publié actuel.
3. Les lignes `maximus_company_modules` sont créées ou mises à jour.
4. Le compte administrateur entreprise est créé ou rafraîchi.
5. La société et la demande passent à l'état actif/approuvé.
6. Une installation dédiée peut ensuite être émise avec une identité, des domaines et un protocole compatibles.

7.3 Mise en place de l'organisation

Créer la hiérarchie → configurer les rôles → créer les employés → attribuer les managers → vérifier les accès

- Les unités sont reliées par `parentId`.
- Les employés sont rattachés à une unité/secteur et à un rôle.
- La création d'un compte interne passe par l'API serveur avant la mise à jour locale.
- Un changement de rôle ou de périmètre doit synchroniser les permissions du compte authentifié et révoquer les sessions si nécessaire.
- La suppression est une suspension serveur ; la révocation de session précède la disparition locale.

7.4 Exploitation et maintenance

MAXIMUS peut placer un module en **ACTIF**, **BETA**, **MAINTENANCE** ou **INACTIF**. La maintenance est immédiate et par entreprise/module : les appels opérationnels renvoient 503, tandis que MAXIMUS conserve son accès technique.

Limite : aucune fenêtre de maintenance planifiée avec début/fin n'a été trouvée. Le statut est un interrupteur immédiat, pas un calendrier.

8. Espaces, routes et navigation

Espace	Routes canoniques	Contenu
Connexion	/, connexion entreprise personnalisée	Session, restauration, déconnexion, liens de création.
Inscription	/inscription	Catalogue, secteur, packs, fonctionnalités, demande manuelle ou intelligente.
MAXIMUS	/maximus/...	Dashboard, entreprises, demandes, catalogue, secteurs, abonnements, wallet, contrôle, journal, santé, paramètres.
Entreprise/employé	/entreprise/...	Dashboard, organisation, contrôle, Commerce, Stocks, Présences, Paie, E-commerce, Transport, rapports.
Compatibilité	/kora/...	Alias historique ; les nouvelles routes utilisent /entreprise/...
Public	/shop/... et domaines personnalisés	Vitrine, catalogue, compte client, panier, checkout, suivi, Transport public et PWA.

Les onglets passent par la query string : par exemple `/entreprise/stocks?tab=inventory`. Les routeurs comparent le chemin sans query string, ce qui évite qu'un onglet interne fasse sortir la route de son espace.

9. Modules métier détaillés

9.1 Gestion commerciale

Le module gère les clients, ventes, produits, fournisseurs, achats, dépenses, caisse, crédit, factures, retours, rapports et activité. Les packs consultation, gestion, employé et manager permettent de réduire le périmètre. Les droits d'un employé commercial sont distincts de ceux d'un manager.

Règle : l'onglet ou la route visible ne suffit pas ; les permissions Commerce doivent être recalculées depuis le rôle, l'unité et le plafond entreprise.

9.2 Gestion de stock

Le serveur expose produits, fournisseurs, entrepôts, emplacements, mouvements, demandes, inventaires et validation. Les ressources doivent appartenir à l'entreprise ; un emplacement doit appartenir au bon entrepôt. Les opérations sensibles demandent une confirmation frontend, mais l'API reste la protection réelle.

À surveiller : tester la concurrence lors de deux mouvements ou validations simultanés, les stocks négatifs, l'idempotence et la cohérence du solde. La surface de route ne prouve pas à elle seule l'intégrité concurrente.

9.3 Présences

Le module gère tableau de bord, pointage, présences, absences, horaires, congés, historique et rapports. Le pointage applique une machine d'état arrivée → sortie, avec pauses selon le modèle. Le QR est signé, borné à l'entreprise/secteur et expiré par date.

- Un employé ne pointe que pour lui-même.
- Un manager est borné à ses secteurs.
- La génération QR est réservée aux acteurs habilités.

- L'export CSV est actuellement généré côté client et doit être considéré comme peu auditable.

9.4 Paie

Le parcours est : bénéficiaires → brouillon de lot → soumission → validation → alimentation du solde → virement groupé → historique. Les comptes bancaires sont chiffrés dans les éléments de lot ; les montants sont positifs et bornés ; l'idempotence est préparée pour les paiements.

Point critique : l'appel à un prestataire externe et la transaction locale n'ont pas une atomicité globale. Les webhooks, idempotences et réconciliations doivent rester testés contre les doubles appels et les réponses tardives.

9.5 Contrôle & coordination

Le flux central est **événement** → **tâche** → **approbation** → **mise à jour** → **audit** → **notification**. Les tâches, événements et audits sont bornés par entreprise, unité et affectation. C'est le socle de traçabilité transverse, pas un moteur complet de workflow arbitraire.

9.6 Modules opérationnels

Le registre frontend contient des modules opérationnels supplémentaires (finance, achats, RH, rapports et autres vues). Ils doivent être distingués des modules Laravel effectivement activés : un identifiant présent dans le registre de navigation n'est pas automatiquement un droit serveur publié.

9.7 Assistant MAXIMUS

L'assistant doit répondre depuis un contexte filtré par session, entreprise et modules autorisés. Une action de mutation doit rester explicite, contrôlée et auditable ; l'assistant ne doit pas transformer une suggestion en mutation implicite.

10. E-commerce, boutique publique et PWA

10.1 Configuration de la boutique

L'administrateur configure nom, slug, description, statut, devise, logo, images de bannière, couleurs, catégories, produits, produits numériques, locations, domaines, zones de livraison et paramètres publics. Les couleurs de boutique restent isolées du thème global entreprise.

10.2 Produit physique et numérique

- Les produits physiques ont prix, stock, images, catégories et état de publication.
- Les produits numériques sont créés en brouillon, reçoivent leur fichier, puis peuvent être publiés.
- Le téléchargement n'est visible qu'après paiement confirmé ; il ne dépend pas d'une URL optionnelle mal formée.
- Le panier impose une unité pour les produits numériques et conserve le type de fulfillment.

10.3 Parcours client

Vitrine → catalogue → panier → compte/adresse → commande → checkout DiamanoPay → webhook/confirmation → fulfillment

- Les sessions clients sont séparées des sessions internes MAXIMUS.
- Panier, commandes, adresses et favoris sont bornés par le client et l'entreprise.
- La commande est verrouillée pendant la création du checkout ; une charge PENDING existante est réutilisée.
- Le paiement confirme la commande ; le webhook est protégé par signature HMAC.
- Les zones de livraison actives sont bornées à l'entreprise et leur identité/tarif est figé dans la demande.

10.4 Commission et portefeuille vendeur

La politique par défaut est 3 % DiamanoPay, 2 % MAXIMUS et 95 % vendeur. Une vente payée peut être réconciliée séparément pour créditer le vendeur et le registre MAXIMUS sans double écriture. Le solde vendeur devient retirable après livraison ou maturation de sept jours sans litige selon la politique.

10.5 PWA

Le service worker met en cache le shell et les ressources GET, mais contourne les routes API. La boutique est donc installable et résiliente pour son interface, pas une application offline complète pour les données, le checkout ou les mutations.

Risques principaux : les images de logo/produit et certaines images Transport sont persistées en base64 ; cela augmente la taille de la base et des réponses. Les médias publics sont volontairement accessibles publiquement : ils ne conviennent pas à des documents confidentiels.

11. Transport public / Taxi

11.1 Configuration interne

Le module configure chauffeurs, disponibilité, GPS, véhicules, tarifs normaux/tempête, paramètres et galerie de couverture. Le chauffeur est créé dans Organisation puis reçoit les informations Taxi spécifiques ; le module Transport ne doit pas devenir une seconde source d'identité RH.

11.2 Parcours public

Position GPS client → lieux/Dakar → devis routier → course → rapprochement chauffeur libre → offre → acceptation → suivi → fin/annulation

- Le périmètre actuel est Dakar, avec bornes géographiques codées.
- Le devis doit utiliser le géocodage puis la matrice OpenRouteService ; aucune distance inventée ne doit servir de fallback.
- Le serveur choisit un chauffeur actif, disponible et géolocalisé récemment.
- Le client conserve un identifiant de course et un jeton d'annulation pour restaurer la course après navigation ou erreur réseau.
- La course ne doit être nettoyée du stockage navigateur qu'après un état terminal confirmé.
- Le thème Transport public reste indépendant de la boutique et du thème entreprise.

11.3 Limites et contrôles

- Le GPS est nécessaire dans le parcours actuel ; aucun parcours manuel complet hors Dakar n'est établi.
- Le jeton public réduit le risque d'utiliser le seul identifiant, mais il doit rester non devinable, expiré et borné à la boutique.
- Les appels publics sont ralentis/throttlés ; les tests de rejeu, expiration, cross-tenant et polling doivent être maintenus.
- Les tuiles Leaflet peuvent échouer ; le trajet doit rester exploitable même sans fond cartographique.

12. Paiements, commissions et frontière financière

12.1 DiamanoPay

Le service supporte une configuration par token statique ou OAuth, webhook secret, timeout, retry et clé d'idempotence. Les credentials restent côté serveur. La création d'un checkout ne doit jamais être simulée côté frontend.

12.2 Règle de crédit

Règle : aucune rémunération vendeur, aucun virement Paie et aucun retrait ne doit être crédité sur la seule base d'un clic de retour frontend. Il faut une confirmation serveur : webhook signé, statut confirmé, commande verrouillée et écriture idempotente.

12.3 Limites observées

- La transaction PostgreSQL ne peut pas annuler une opération déjà acceptée par le prestataire externe.
- Les webhooks et la réconciliation doivent couvrir les appels dupliqués, les retards et les divergences de statut.
- Aucun flux complet de remboursement, litige ou chargeback n'a été identifié dans les routes inspectées.
- Les règles d'abonnement décrivent des plans, limites et prix dans le frontend, mais ne constituent pas encore une facturation serveur complète.

13. Données, API et persistance

13.1 PostgreSQL Laravel

Les migrations Laravel couvrent l'identité, les sessions, le contrôle, les présences, les stocks, les accès modules, l'app-state, l'e-commerce, les clients, le portefeuille vendeur, la paie, les domaines, le Transport, les installations et les réglages. La base Render est la source durable du runtime actif.

13.2 App-state et données organisationnelles

Les collections organisationnelles (unités, rôles, employés métier, abonnements descriptifs, catalogue workspace) sont conservées dans un état versionné avec verrouillage de ligne et contrôle de conflit. Les comptes d'authentification, sessions et accès modules sont plus durablement structurés côté serveur.

Écart de durabilité : la récupération d'état peut reconstruire entreprises, employés, rôles et unités depuis les comptes et accès modules, mais les abonnements sont explicitement reconstruits comme vides. Ils ne doivent donc pas être considérés comme un registre financier durable.

13.3 LocalStorage et fallback

Le navigateur garde session d'interface, préférences, état de menu, recherche, certains états Commerce, et l'état de restauration d'une course Taxi. Ce stockage est utile pour l'ergonomie et la reprise, mais ne doit pas être présenté comme preuve de sauvegarde serveur. Un fallback local peut masquer une panne de persistance si l'écran ne signale pas clairement son mode dégradé.

13.4 Routes Laravel

Fichier	Famille
routes/api.php	Santé, auth, inscription, entreprises, modules, installations, contrôle central, paramètres et portefeuille.
routes/ecommerce.php	Boutique interne/public, produits, commandes, clients, checkout, digital, location, livraison, wallet.
routes/transport.php	Chauffeurs, véhicules, courses, settings, devis, lieux, suivi et endpoints publics.
routes/presence.php	Présences, QR, pointage direct et scan.
routes/stock.php	Référentiels, mouvements, demandes, inventaires et validation.
routes/payroll.php	Bénéficiaires, lots, validation, solde, virement et historique.
routes/control.php	Tâches, événements et audit Contrôle & coordination.

14. Sécurité, isolation et installations

14.1 Protections présentes

- Sessions serveur avec jetons stockés hachés et cookie HttpOnly.
- Hachage des mots de passe, migration transparente des anciens hash après validation.
- Contexte entreprise imposé côté serveur.
- Autorisation module/fonctionnalité/action côté contrôleurs.
- QR Présences signé avec date, entreprise et périmètre.
- Webhook de paiement signé HMAC.
- Limites de type/taille sur les téléversements.
- En-têtes de sécurité : CSP, nosniff, DENY framing, Referrer-Policy et HSTS en production.
- Verrou de suppression entreprise contrôlé en transaction et renvoyant 423.

14.2 Points qui nécessitent une décision ou une vérification

Point	Constat	Impact possible
CSRF	Cookie SameSite=Lax ; aucun flux CSRF dédié n'est clairement documenté dans l'audit du code.	À vérifier pour toutes les mutations API basées sur cookie.
MFA / mot de passe oublié	Aucune route complète visible pour MFA ou réinitialisation.	Risque de récupération de compte et de sécurité opérationnelle.
CSP	style-src unsafe-inline et images https larges.	Surface XSS/supply-chain plus permissive ; à durcir si compatible.
Médias base64	Images produit et Transport parfois stockées en base.	Volumétrie, sauvegardes et temps de réponse.
Token public	Course et annulation utilisent une preuve publique restaurable par navigateur.	Tester expiration, replay, devinabilité et isolation domaine.

14.3 Installations dédiées

Le mode central reste le défaut. Une installation dédiée est bornée par une entreprise configurée côté serveur, son hôte, son token et son protocole. Elle ne crée jamais d'administrateur global. La synchronisation centrale vérifie l'identité et la compatibilité avant de mettre à jour les modules/domaine.

15. Déploiement et exploitation

15.1 Render central

- Service Docker web maximus-erp.
- Health check : /api/healthz.
- PostgreSQL Render géré, DB_CONNECTION=pgsql.
- Session et cache en base.
- Disque persistant /var/data pour fichiers numériques.
- Entrypoint : validation des variables, migrations avec retry, provisionnement admin central ou installation dédiée, puis Apache.

15.2 Installation locale/dédiée

Les scripts d'installation appliquent les migrations, construisent le frontend, créent le compte company_admin, activent la synchronisation centrale et conservent les fichiers persistants. Ils ne créent pas de maximus_admin dans une installation entreprise.

15.3 Discipline de livraison

- 1. Typecheck workspace et artefacts.
- 2. Tests frontend et Laravel.
- 3. Build avec PORT et BASE_PATH.
- 4. Redémarrage du workflow concerné.
- 5. Logs workflow et console navigateur.
- 6. Vérification des routes authentifiées et publiques qui ont changé.
- 7. Ne jamais pousser vers GitHub sans autorisation explicite.

16. Tests et niveau de confiance

16.1 Couverture existante

- Tests frontend dans `artifacts/maximus/src/lib/*.test.ts` : permissions, navigation, catalogue, app-state, PWA, Commerce, Transport, installation, assistant et mutations.
- Tests Laravel Feature : auth, contrôle, catalogue, inscription, app-state recovery, installations, e-commerce, digital, wallet vendeur, paie, présences, stock, Transport, sécurité.
- Audits précédents : une suite intégrée documentée à 32 tests/203 assertions côté Laravel et 147/147 côté frontend pour un périmètre catalogue/inscription ; une vérification de navigateur a également couvert des fixtures synthétiques.
- Dernière vérification du changement de couleurs : TypeScript réussi, 149 tests frontend réussis, build Vite réussi et workflow MAXIMUS redémarré.

16.2 Ce qui n'est pas prouvé par les tests actuels

Parcours	État de preuve
Login → refresh → logout pour chaque rôle et installation	À compléter en E2E
Checkout public → retour → webhook → commande → commission	À compléter avec sandbox prestataire
Double webhook, double checkout, réconciliation et retrait	À renforcer
Installation PWA, cache, reprise réseau et version de shell	À compléter
GPS, permissions navigateur, expiration course, annulation et cross-tenant	À compléter
Upload à la limite, médias publics, stockage long terme et volume DB	À compléter
Concurrence Stock/inventaire et validation Paie	À renforcer
CSRF, MFA, récupération de compte et durcissement CSP	Décision + vérification sécurité

17. Constats priorités et plan recommandé

Priorité	Constat	Pourquoi	Action recommandée
P0	Renouvellement/abonnement réel non matérialisé côté serveur.	Le modèle frontend ne suffit pas à bloquer ou facturer un accès.	Décider la source d'abonnement, créer le registre durable, le cycle de renouvellement, les webhooks et les tests.
P0	Parité Laravel/PostgreSQL avec Express/Drizzle non totalement prouvée.	Deux frontières de données augmentent le risque d'écart de comportement.	Finaliser la preuve de parité avant suppression ou changement de responsabilité.

P1	Paiements et retraits sans matrice E2E complète.	Les side effects externes ne sont pas annulables par une transaction locale.	Tester webhooks signés, doubles appels, retards, divergences, commission et maturation.
P1	CSRF/MFA/récupération de compte à clarifier.	Les sessions cookie et comptes métier portent des opérations sensibles.	Faire une revue sécurité dédiée et décider les contrôles obligatoires.
P1	Images base64 en base.	Risque de volumétrie, sauvegarde lente et réponses lourdes.	Basculer les ressources persistantes vers le stockage objet/disque durable avec métadonnées en base.
P1	Maintenance sans fenêtre planifiée.	Un statut immédiat ne permet pas d'organiser un changement à l'avance.	Ajouter début/fin, message opérateur, fuseau et retour automatique si nécessaire.
P2	Pas de matrice E2E complète des surfaces publiques.	Les routes boutique, domaines, PWA et Taxi sont nombreuses.	Créer une suite navigateur synthétique multi-entreprises et multi-rôles.
P2	Commerce interne encore en repli local/prototype.	Un succès d'interface peut masquer une absence de persistance.	Afficher le mode dégradé et finaliser le cycle serveur ou le déclarer explicitement.
P2	Export Présences client-side et absence de chargeback détecté.	Traçabilité et opérations financières peuvent être incomplètes.	Décider export serveur audité et politique de remboursement/litige.

Ordre conseillé

1. Verrouiller la source de vérité Laravel/PostgreSQL et la facturation/abonnement.
2. Finir la campagne de tests financiers et de séparation de tenants.
3. Durcir les comptes et sessions : CSRF, MFA, récupération, rotation et audit.
4. Déplacer les fichiers lourds hors base et vérifier la capacité de production.
5. Ajouter les E2E des parcours qui touchent argent, identité, domaine public et GPS.
6. Seulement ensuite traiter les améliorations UX et les extensions de catalogue.

18. Référentiel pratique : quoi, comment, où

Besoin	Point d'entrée	Source d'autorité
Ajouter un module	ModuleCatalog.php, registre frontend, routes, permissions, tests.	Catalogue publié + accès entreprise Laravel.
Ajouter un pack	Définition feature_packs, permissions, sélection, normalisation, tests.	Catalogue publié ; un pack ne crée aucune identité arbitraire.
Ajouter un secteur	Preset secteur et ses propositions module/fonctionnalité.	Profil de proposition ; pas une autorisation directe.
Changer un accès entreprise	Contrôleur module/entreprise et table maximus_company_modules.	Laravel + rôle MAXIMUS.
Créer un employé	Organisation → comptes/managers → /api/auth/accounts.	Compte/auth serveur, puis app-state organisation.
Protéger une fonctionnalité	Identifiant canonique, ModuleAuthorization, route/controller et UI.	Serveur ; l'UI ne fait que refléter.
Modifier la boutique	ecommerce-module.tsx et routes/ecommerce.php.	Entreprise + module e-commerce + feature/action.

Modifier le Transport	transport-module.tsx et routes/transport.php.	Entreprise + module Transport + permission + ressources tenant.
Modifier les Présences	presence-module.tsx et routes/presence.php.	Entreprise, employé/secteur, feature et état du pointage.
Déployer	render.yaml, Dockerfile.render, render-entrypoint.sh.	Render + PostgreSQL + variables d'environnement gérées.
Vérifier avant livraison	Typecheck, tests, build, logs, preview et routes modifiées.	Commandes documentées dans README.md.

19. Sources de l'audit

- README.md — architecture, runtime, installation, déploiement et règles de livraison.
- docs/technical.md — structure frontend, état, permissions, catalogue, organisation, API et tests.
- docs/audits/company-catalogue-registration-audit.md — limites et vérifications du catalogue/inscription.
- artifacts/maximus/src/App.tsx, src/routes/app-routes.tsx, src/lib/* — session, routes, accès, navigation et tests frontend.
- artifacts/api-server/laravel/app/Support/ModuleCatalog.php — modules, fonctionnalités, packs et dépendances.
- artifacts/api-server/laravel/app/Http/Middleware/* — session, tenant, installation et accès modules.
- artifacts/api-server/laravel/routes/*.php — surface API active.
- artifacts/api-server/laravel/database/migrations — modèle PostgreSQL Laravel.
- artifacts/api-server/laravel/tests/Feature et artifacts/maximus/src/lib/*.test.ts — couverture de tests.
- render.yaml, Dockerfile.render, render-entrypoint.sh — exploitation Render.

À retenir : MAXIMUS est organisé pour permettre à MAXIMUS de publier des modules, packs et secteurs, puis à chaque entreprise de composer son espace sans modifier le code. La règle de sécurité non négociable est que le serveur Laravel doit rester la dernière instance de décision pour l'identité, l'entreprise, le module, la fonctionnalité, l'action et la propriété de la donnée.

Document généré depuis le snapshot de travail du 22 septembre 2026. Les chemins et constats doivent être revalidés après une migration majeure ou un changement de runtime.